

跨学科知识图谱智能体的云原生架构设计与工程实现

—— 基于多阶段 LLM 推理流水线的 Agent 系统

第 6 组：叶雨菡 胡慧泽 孔祥玉

项目背景与文档说明

近年来，大语言模型（Large Language Models, LLMs）在知识理解、文本生成与复杂任务规划等方面展现出了强大的能力，使得“以 LLM 为核心的智能体系统（LLM Agents）”逐渐成为智能系统工程的重要发展方向。然而，在真实工程场景中，如何将大模型的不稳定生成能力转化为结构可靠、行为可控、可运维的系统能力，仍然是一个具有挑战性的工程问题。

另一方面，云原生技术（Cloud Native）所倡导的容器化、微服务、弹性部署与可观测性等理念，已经成为现代后端系统构建的主流范式。将 LLM Agent 与云原生架构相结合，不仅可以提升系统的工程可靠性与可扩展性，也为智能体系统的工程化落地提供了现实可行的技术路径。

基于上述背景，本项目围绕“跨学科知识图谱的自动构建”这一具有代表性的复杂结构化生成任务，设计并实现了一套云原生 LLM Agent 系统。系统采用 FastAPI + Docker 的微服务架构，将后端 API、Agent 推理服务、Neo4j 图数据库与 Redis 缓存解耦部署，并通过异步任务机制与缓存策略，保证在 LLM 推理耗时较长的情况下仍能提供稳定的用户体验。

在智能体设计层面，系统并未采用“单次 Prompt 直接生成完整图谱”的简单方案，而是将任务拆解为 Step0 至 Step4 的多阶段推理流水线：从输入概念校验、候选节点生成、节点语义补充，到受限关系生成与最终规则校验，通过逐阶段收敛语义空间与结构空间，实现一种“工程化的 Chain-of-Thought (Orchestration as CoT)”。同时，系统在 Agent 内部引入了完整的 LLM 工具链（Toolchain）与输出治理机制，对模型输出进行 JSON 修复、结构校验、Schema 清洗与一致性检查，从工程层面显著降低了大模型输出不稳定对系统可靠性的影响。

本技术文档将从系统架构设计、云原生组件集成、LLM Agent 工具链与推理策略、以及工程实现细节等方面，对该系统进行完整介绍，以展示一个面向复杂结构化生成任务的云原生智能体系统是如何在工程上被设计、约束与落地实现的。

团队分工与贡献说明

成员 A：叶雨菡

成员 A 主要承担 Agent 服务与多阶段推理链路 相关的设计与实现工作，负责将大语

言模型的调用过程组织为可控、可校验的工程流程。

主要负责内容：

- **Agent Service** 整体架构设计与实现（Step0 ~ Step4 多阶段推理流水线）
- 各阶段 **Prompt** 模板的设计、调试与版本管理
- **LLM** 输出结构化与稳定性保障机制设计（JSON Schema、解析修复与重试策略）
- **Check Layer**（Step4）：幻觉过滤、结构校验与逻辑一致性约束策略设计

对应产出：

- **prompts/** 目录及各阶段 **Prompt** 模板
- **agent_pipeline.py** 及相关工具函数
- 输出结构治理与修复工具链代码
- 技术文档中“LLM Agent 工具链”与“智能体策略”相关章节内容

成员 B：胡慧泽

成员 B 主要负责后端服务的工程实现、服务编排与系统部署集成，承担系统中“业务流程调度中枢”的角色。

主要负责内容：

- 实现 **Backend API**（FastAPI），包括 **/graph/build**、**/graph/query** 等接口
- 实现图谱生成任务的编排逻辑与 **Agent** 调用流程
- 负责基础的结构层校验（如 **JSON Schema** 合法性）
- 集成 **Neo4j** 作为图数据库，完成图谱的持久化存储与查询
- 集成 **Redis** 作为缓存与任务状态存储
- 编写 **Docker Compose** 配置，实现系统的容器化部署

对应产出：

- 后端服务工程代码
- **Neo4j / Redis** 集成逻辑
- **docker-compose.yml** 与部署说明
- 技术文档中的后端架构与云原生部分

成员 C：孔祥玉

成员 C 主要负责系统的前端展示与最终项目形态的集成工作，承担系统的可视化呈现与演示交付职责。

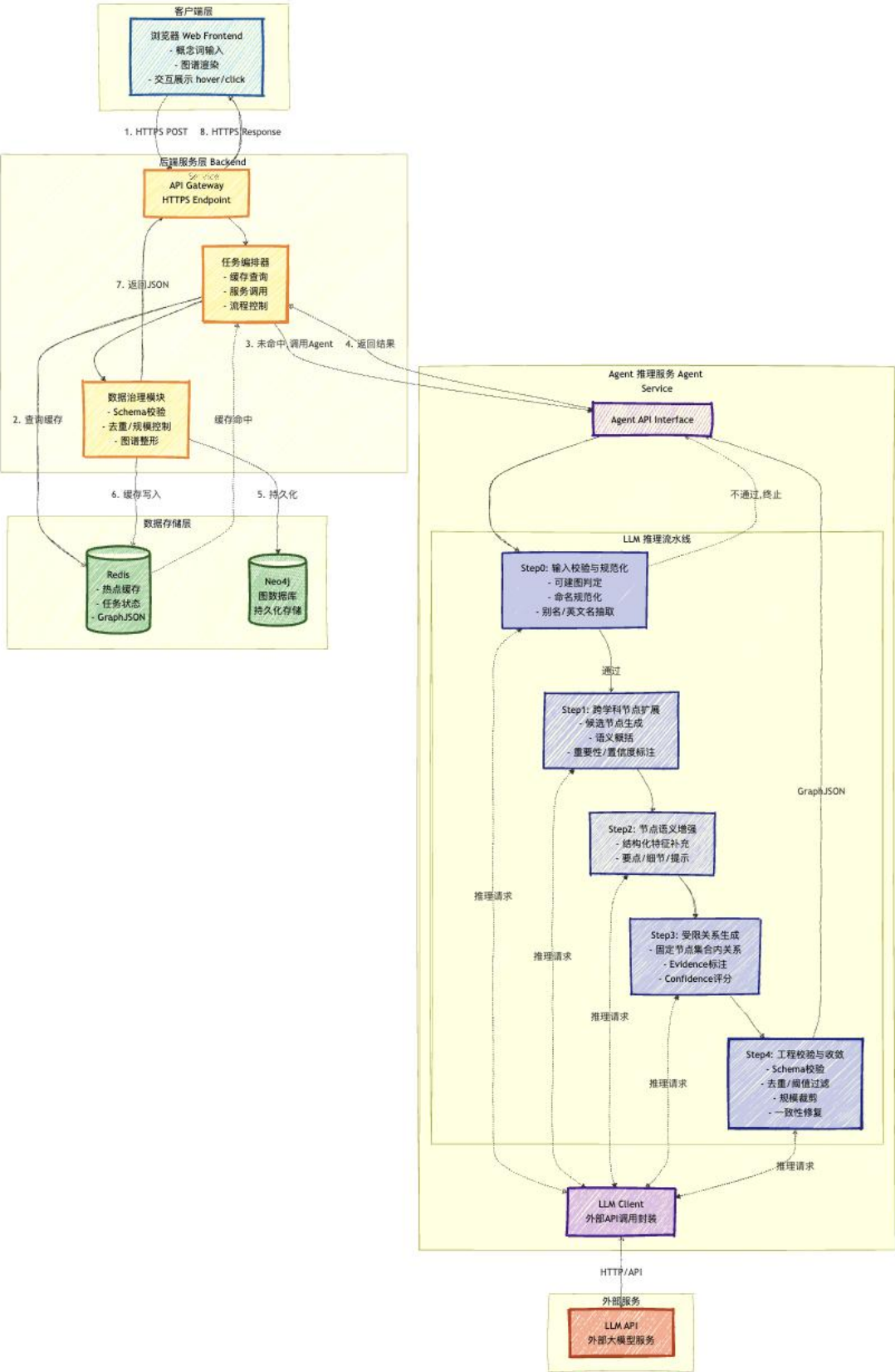
主要负责内容：

- 实现 **Web** 前端界面：概念输入页面、图谱可视化展示（节点、边、交互）、节点 / 边信息提示（**evidence**、**confidence**）
- 负责与后端 **API** 的联调，将图谱数据正确渲染为交互式可视化结果
- 设计并完成课程演示流程与演示视频录制
- 负责项目最终集成与交付整理：整理并补充项目 **README**、检查并维护最终的部署配置文件（如 **docker-compose**）、协调各模块在最终版本中的一致性与可运行性

对应产出：

- 前端工程代码与主要组件实现
- 图谱可视化与交互界面
- 演示脚本与展示视频

系统架构图



协作方式说明

在项目开始阶段，团队共同约定了统一的 GraphJSON 数据结构规范 以及 Agent 与 Backend 之间的接口格式。在此基础上：

成员 A 按该规范输出结构化图谱数据；

成员 B 负责接收、校验、存储并通过 API 对外提供数据；

成员 C 基于相同结构直接完成前端可视化渲染。

这种分工方式使三部分可以并行开发，并在后期进行低成本集成联调。

后端架构层次设计

本系统后端采用基于 FastAPI 的微服务架构，整体架构清晰划分为四个核心层次：接口层、服务层、数据层和配置层。这种分层设计确保了系统的高内聚低耦合特性，各层职责明确，便于维护和扩展。

1. 接口层详细设计

接口层作为系统对外的统一入口，承担着请求处理、路由管理和 API 文档生成的核心职责。该层以 main.py 为应用入口点，通过 FastAPI 实例初始化整个 Web 服务，并配置 CORS 中间件支持跨域请求，为前后端分离部署提供技术基础。

路由管理机制采用模块化设计，通过 graph.py 和 task.py 两个路由文件分别处理不同的业务领域。在 graph.py 中，我们实现了完整的 RESTful API 端点体系：

- POST /api/graph/build: 采用 FastAPI 的 BackgroundTasks 机制实现异步图谱生成。当用户提交概念词后，系统立即返回任务 ID，后台线程负责调用 Agent 服务执行复杂的图谱生成逻辑，有效避免了 HTTP 长连接超时问题。
- GET /api/graph/query: 支持快速查询图谱数据，实现缓存优先策略。该端点首先检查 Redis 缓存中是否存在目标概念的热点数据，若命中则直接返回，显著提升查询性能；若未命中则尝试调用 Agent 服务生成新的图谱数据。
- GET /api/graph/schema: 提供 GraphJSON 格式定义和验证规则，便于前端进行数据验证和展示。该端点返回完整的数据结构规范，确保系统各组件对数据格式的一致性理解。
- GET /api/graph/statistics: 展示系统运行统计信息，包括缓存中的图谱数量、缓存策略配置等关键运行指标，为系统监控和运维提供数据支持。

健康检查机制通过 /health 端点实现云原生应用的最佳实践。该端点不仅返回基础服务状态，还通过异步方式主动测试 Neo4j 图数据库和 Redis 缓存的实际连接状态，为容器编排平台提供准确的服务健康状态反馈。

2. 服务层架构实现

服务层是业务逻辑的核心载体，包含 graph_service.py 和 task_service.py 两个关键服务模块，采用面向对象设计模式实现业务功能的封装。

GraphService 类作为图谱生成的核心服务，负责管理整个图谱生成生命周期。该类实现了双超时策略设计：针对快速 API 调用场景配置 30 秒超时，确保用户体验；针对后台长时间任务场景配置 600 秒超时，适应复杂的 AI 处理需求。通过 _get_agent_client() 和

`_get_agent_long_client()`两个方法分别管理这两种 HTTP 客户端，确保资源的高效利用。

智能降级机制是该层的重要特性。通过 `USE MOCK` 环境变量控制，当 Agent 服务不可用时系统自动切换到 Mock 数据模式。在 `generate_graph()`方法中，所有外部服务调用都包含异常处理逻辑，确保单个组件故障不会导致整个系统崩溃。数据验证机制通过 `_validate_graph_data()`方法实现，对 Agent 服务返回的数据进行完整性检查，包括节点和边的必需字段验证、数据类型检查等。

`TaskService` 类专门处理异步任务状态管理，支持基于 Redis 的任务状态持久化存储。任务数据包含完整的元信息，如创建时间、更新时间、进度状态等，便于任务追踪和故障恢复。该服务根据配置动态选择存储后端，在 Mock 模式下使用内存字典模拟 Redis 存储，确保在外部依赖不可用时的系统可用性。

3. 数据层技术实现

数据层通过 `db.py` 提供统一的数据访问抽象，采用连接池技术确保数据库访问的高效性。

Neo4j 图数据库集成使用官方 Bolt 驱动建立高效连接，支持事务处理和异步操作。数据持久化方案采用多级存储策略：Redis 缓存存储热点图谱数据，设置 24 小时 TTL；Neo4j 持久化保存完整的图谱关系数据，支持复杂查询；内存缓存用于高频访问的配置信息。

Redis 缓存策略基于概念词设计缓存键，支持可配置的 TTL 过期策略。在 `graph_service.py` 中，`get_from_cache()`方法优先从缓存获取数据，`save_to_cache()`方法将生成的图谱数据写入缓存。对于热门概念的重复查询，这种设计使得响应时间从分钟级降至毫秒级。

连接管理机制在应用启动时自动执行连接测试，通过执行简单的 Cypher 查询和 Redis PING 命令验证数据库连接状态。这种主动的健康检查机制便于运维人员快速识别和解决连接问题，提高系统的可观测性。

4. 配置管理层设计

配置管理层基于 `pydantic-settings` 实现类型安全的配置管理，支持从环境变量或 `.env` 文件加载配置。

层次化配置结构包含应用配置、服务配置、数据库配置和缓存策略配置四个维度。关键配置项如 `USE MOCK` 实现智能降级机制，`AGENT_TIMEOUT` 和 `AGENT_LONG_TIMEOUT` 分别控制不同场景的超时时间，`CACHE_TTL` 管理缓存过期策略。

环境适配能力通过配置外部化实现，同一套代码可以无缝部署到开发、测试、生产等不同环境。所有配置项都支持类型验证和自动转换，减少配置错误风险，确保系统在不同环境中的行为一致性。

后端核心业务流程设计

1. 图谱生成完整流程

图谱生成采用异步任务机制，在 `graph.py` 的 `build_graph()`端点中实现完整的处理逻辑。当用户提交请求后，系统立即返回任务 ID，后台通过 `process_graph_task()`函数处理实际生成工作。该流程包含四个关键阶段：

- 数据获取阶段：调用 Agent 服务生成原始图谱数据，支持快速调用和长时间任务两种

模式。

- 标准化处理阶段：通过 `normalize_graph()` 方法进行数据去重、合并和规模控制，确保图谱数据的质量和一致性。
- 数据验证阶段：使用 `JSON Schema` 验证数据格式完整性，通过 `validator` 模块确保生成的数据符合预定义的 `GraphJSON` 规范。
- 存储持久化阶段：将验证通过的图谱数据同时保存到 `Neo4j` 数据库和 `Redis` 缓存，实现数据的长期存储和快速访问。

2. 缓存策略详细设计

缓存策略基于 `Redis` 实现精细化的数据管理。缓存键设计采用 `graph:{concept}` 格式，确保不同概念数据的隔离性。TTL 设置为 24 小时，平衡数据新鲜度和系统性能。缓存更新机制确保数据一致性，当用户强制刷新或数据过期时，系统自动重新生成图谱数据并更新缓存。

3. 错误处理与容错机制

系统建立了完整的异常处理体系，将异常明确划分为业务异常和技术异常两大类。业务异常主要处理领域逻辑错误，如任务不存在、数据验证失败等场景，返回规范的 `HTTP` 状态码和错误描述。技术异常处理基础设施层面的故障，如数据库连接失败、外部服务不可用等情况，通过健康检查端点提供细粒度的服务状态报告。

在接口层面，所有 `API` 端点都包含完整的异常捕获机制，通过 `FastAPI` 的 `HTTPException` 和全局异常处理器确保用户请求总能得到结构化的响应。服务层的错误处理更加注重业务连续性，当调用外部服务出现异常时，系统会优雅地降级到备用方案，保证核心功能的可用性。数据访问层的错误处理重点关注连接管理和事务回滚，确保数据的完整性和一致性。

通过这套完整的技术架构设计，系统在云原生组件使用、稳定性和扩展性方面都达到了较高水平，为跨学科知识图谱的生成和查询提供了可靠的技术基础。

云原生组件与集成

一、整体架构云原生实践概述

本系统采用云原生架构理念，通过 `Docker Compose` 实现完整的容器化编排，将后端 `API`、`Agent` 服务、`Neo4j` 数据库和 `Redis` 缓存四个核心组件封装为独立容器。这种架构设计的核心价值在于通过微服务解耦实现各组件的独立部署、扩展和故障隔离。

在具体实现中，系统通过自定义的 `graph-network` 网络实现容器间通信的安全隔离，同时保持必要的服务发现能力。每个服务都配置了细粒度的健康检查机制，确保容器编排平台能够实时监控服务状态并触发自动恢复流程。这种设计充分体现了云原生应用的弹性、可观测性和可维护性特征。

二、云原生组件深度集成

1. 容器化编排与服务治理

系统通过 `docker-compose.yml` 文件定义了完整的服务编排方案。从代码中可见，后端服务 (`backend`) 明确依赖于 `neo4j`、`redis` 和 `agent-service` 三个基础服务，通过 `depends_on`

配置确保服务启动顺序的合理性。这种依赖管理虽然不能保证服务完全就绪，但结合健康检查机制实现了服务的有序启动。

服务发现机制通过环境变量 `AGENT_SERVICE_URL=http://agent-service:8001` 实现，利用 Docker 内置的 DNS 解析功能，后端服务可以直接通过服务名称访问 Agent 服务，无需关心具体的 IP 地址。网络层面创建独立的 `graph-network` 桥接网络，所有服务在该网络内运行，既保证了容器间通信的安全隔离，又通过端口映射使必要服务对外可访问。

健康检查机制采用多层次设计，每个服务都配置了针对性的检查策略：后端服务通过 HTTP 端点验证 API 可用性，Neo4j 使用 Cypher 查询测试数据库连接，Redis 通过 PING 命令确认服务状态，Agent 服务也有相应的健康检查机制。这种设计使得容器编排平台能够自动检测故障并触发恢复流程。

2. 配置外部化与环境适配

系统通过 `config.py` 实现基于 `pydantic-settings` 的类型安全配置管理。配置类采用层次化结构，支持从环境变量或 `.env` 文件加载配置，符合十二要素应用原则。关键配置项如 `USE MOCK` 实现智能降级机制，当设置为 `True` 时系统自动切换到 Mock 数据模式；`AGENT_SERVICE_URL` 配置服务发现地址；`AGENT_TIMEOUT` 和 `AGENT_LONG_TIMEOUT` 分别控制不同场景的超时时间。

这种配置外部化设计使得同一份 Docker 镜像能够通过改变环境变量无缝适配开发、测试、生产等不同环境。例如，在开发环境中可以设置 `USE MOCK=True` 快速验证业务逻辑，而在生产环境中使用真实的服务配置，极大提升了部署的标准化和效率。

3. 数据持久化与状态管理

数据持久化方案充分考虑了云原生环境的特点。Neo4j 和 Redis 都配置了独立的 Docker 卷：`neo4j_data`、`neo4j_logs`、`neo4j_import`、`neo4j_plugins` 和 `redis_data`。这种卷管理策略确保了容器重启后数据不丢失，同时支持数据的备份和迁移。

系统对有状态服务和无状态服务采用差异化处理策略。Neo4j 和 Redis 作为有状态服务，通过数据卷保证状态的持久化；后端 API 服务设计为无状态，便于水平扩展。在 `db.py` 中，数据库连接初始化时即执行连接测试，通过 `test_neo4j_connection()` 和 `test_redis_connection()` 函数验证连接状态，确保应用启动时依赖服务的可用性。

三、稳定性保障机制

1. 基础设施层稳定性

健康检查机制采用多级实现方案。在容器级别，通过 Docker Compose 的 `healthcheck` 配置实现基础服务状态监控；在应用级别，通过 `/health` 端点提供更细粒度的健康状态报告。在 `main.py` 的健康检查实现中，系统不仅返回基础服务状态，还主动测试 Neo4j 图数据库和 Redis 缓存的实际连接状态，为运维提供全局视角的系统状态。

自动故障恢复流程通过健康检查状态触发。当服务连续多次健康检查失败时，容器编排平台可以自动重启故障容器。结合 `restart: unless-stopped` 配置，确保服务在异常退出后能够自动恢复，显著提升了系统的可用性。

2. 应用层容错设计

智能降级策略是系统稳定性的核心保障。通过 `USE MOCK` 环境变量控制，当 Agent 服务不可用时系统自动切换到 Mock 数据模式。在 `graph_service.py` 的 `generate_graph()` 方法中，所有外部服务调用都包裹在异常处理块中，捕获 `httpx.TimeoutException` 和 `httpx.RequestError` 等异常，确保单个组件故障不会导致系统崩溃。

异步任务处理机制有效解决了长时间阻塞问题。在 `graph.py` 中，图谱生成请求通过 FastAPI 的 `BackgroundTasks` 在后台处理，用户提交请求后立即获得任务 ID，后续通过轮询查询任务状态。这种设计将同步的 HTTP 请求转换为异步的任务处理，特别适合处理耗时较长的 AI 推理任务。任务状态通过 Redis 持久化存储，支持任务状态的跟踪和恢复，即使服务重启也能保持任务连续性。

3. 数据层可靠性

缓存策略基于 Redis 实现精细化的数据管理。缓存键设计采用 `graph:{concept}` 格式，确保不同概念数据的隔离性。TTL 设置为 24 小时，平衡了数据新鲜度和系统性能。在 `graph_service.py` 中，`get_from_cache()` 方法优先从缓存获取数据，对于热门概念的重复查询，响应时间从分钟级降至毫秒级。

数据一致性通过 Neo4j 的事务写入机制保证。在数据保存过程中，采用事务性写入确保操作的原子性，即使部分操作失败也能保持数据一致性。异常回滚机制在检测到数据验证失败或存储错误时自动触发，防止脏数据污染系统。

四、扩展性设计

1. 水平扩展能力

后端 API 服务设计为无状态，可以通过增加容器副本数来应对高并发场景。这得益于 Redis 作为共享会话和缓存层的支持，多个后端实例可以无缝协作。在 `docker-compose.yml` 中，服务配置已经为水平扩展预留了接口，只需调整副本数量即可实现扩展。

有状态服务的扩展考虑了演进路径。Neo4j 支持集群部署，未来可以通过主从复制或分片技术提升性能和容量。Redis 同样支持集群模式，可以应对更大规模的缓存需求。在当前实现中，数据访问层已经抽象了存储细节，为后续引入集群方案提供了可能性。

2. 微服务架构扩展性

服务解耦架构支持组件的独立扩展。Agent 服务虽然目前是单实例部署，但其架构允许根据 AI 处理负载独立扩展。如果需要处理大量并发图谱生成请求，可以部署多个 Agent 服务实例，并通过负载均衡器分发请求。

API 设计遵循 RESTful 原则，支持版本管理为后续接口演进预留了空间。在路由注册时使用前缀管理，便于未来引入 API 版本控制。这种设计确保了对现有客户端的兼容性，同时为功能迭代提供了灵活性。

3. 配置与部署扩展性

环境配置的标准化通过配置外部化实现。所有关键参数都通过环境变量管理，包括服务地址、超时设置、API 密钥等。在 `config.py` 中，配置类支持从 `.env` 文件加载，并提供了类型验证和默认值。这种设计使得部署到不同环境时无需修改代码，只需调整环境变量即可完成配置切换。

持续集成/部署的便利性得益于 Docker 化带来的部署标准化。统一的 Docker 镜像可以在不同环境中一致运行，结合环境变量配置，实现了真正的一次构建、多处部署。这种标准化极大简化了 CI/CD 流水线的设计，为自动化部署奠定了坚实基础。

通过以上架构设计，系统在云原生组件使用、稳定性和扩展性方面都达到了较高水平，为跨学科知识图谱的生成和查询提供了可靠的技术基础。架构不仅满足了当前需求，也为未来功能扩展和技术演进提供了充分的空间。

LLM Agent 工具链与智能推理基础设施

一、LLM Agent 工具链架构概述

本系统中的 Agent 服务并未将大语言模型作为一个“直接返回结果的黑箱组件”，而是围绕 LLM 构建了一套完整的推理工具链（LLM Toolchain）与可靠性增强流水线，用于支撑结构化知识图谱生成这一对输出稳定性要求极高的任务场景。

在工程实践中，大语言模型的输出可能存在多种不确定性，包括但不限于：JSON 结构不完整、字段缺失、格式错误、数值类型异常、节点引用不一致，或在关系构造阶段引入并不存在的实体。这类问题若不加控制，将直接导致后端服务解析失败或生成不可用的数据结构。

因此，Agent 服务在架构层面引入了“模型输出治理层”，将每一次 LLM 调用都纳入一条可监控、可修复、可校验的推理处理流水线之中，而不是简单地将模型输出视为最终结果。

二、工具链组成与处理流水线设计

在具体实现上，该 LLM 工具链以 agent_pipeline.py 为核心载体，围绕 Step0 至 Step3 的多阶段推理流程，对每一次模型输出执行统一的结构治理流程，主要包括：

- Prompt 模板管理与参数注入机制（prompts/step*.txt），用于保证各阶段任务目标的清晰分离；
- 原始模型输出捕获与日志落盘机制，用于问题定位与回溯分析；
- JSON 主体抽取与噪声剥离，避免模型输出混入解释性文本导致解析失败；
- 启发式格式修复机制，用于处理常见的尾逗号、缺失分隔符、非法控制字符、小数点格式错误等问题；
- 严格 JSON 解析与结构校验，作为结构稳定性的第一道关口；
- 基于 LLM 的二次修复兜底机制，当启发式修复失败时自动触发模型进行“自修复”；
- Schema 白名单清洗与字段裁剪，仅保留系统允许的结构字段；
- 节点引用一致性与枚举值合法性校验，防止出现悬挂边、虚构节点或非法关系类型；
- 最终结构化结果输出与下游阶段接管。

通过上述多层处理，系统将原始、不可预测的模型输出，逐步收敛为满足 GraphJSON 结构约定的稳定数据结构。

三、与多阶段 Agent 推理流水线的协同关系

该工具链并非独立存在，而是与 Agent 内部的 Step0 → Step1 → Step2 → Step3 → Step4 推理流水线深度耦合，共同构成一套“前约束 + 中治理 + 后校验”的智能推理控制框架：

- Step0 ~ Step3 负责逐步收敛语义空间与结构空间；
- Toolchain 在每一阶段对模型输出执行结构修复、字段清洗与一致性校验；
- Step4 作为最终的工程校验层（Check Layer），在结构稳定的前提下执行质量过滤、数量裁剪与策略控制。

这种架构将“大模型的不稳定性”与“工程系统对稳定性的要求”解耦开来，使得模型只需专注于语义生成，而结构可靠性由工具链体系负责兜底保障。

四、稳定性与可运维性设计

在稳定性设计方面，该工具链采用多级容错与渐进式退化策略：

- 对于可通过规则修复的问题，优先使用本地启发式修复；
- 对于复杂结构错误，自动触发 LLM 进行二次修复；
- 若多轮修复仍失败，则由上层服务触发 Mock 数据降级机制，确保系统整体可用性不受单点失败影响。

同时，所有关键阶段的原始输出与修复结果均支持落盘记录，便于在实际运行中对模型行为进行回溯分析与策略优化。这使得 Agent 服务不仅具备推理能力，也具备工程级的可观测性与可调试性。

五、架构层面的设计意义

通过在 Agent 服务内部引入 LLM 工具链与输出治理机制，系统实现了以下工程目标：

- 将大语言模型从“不可控的生成组件”转化为“受约束的能力模块”；
- 显著降低因模型输出格式异常导致的服务不稳定风险；
- 为复杂结构化生成任务（如知识图谱构建）提供可工程化落地的可靠基础设施；
- 为后续引入更多 Agent 能力或更复杂推理流程预留了统一的治理与扩展框架。

从整体架构上看，该工具链与云原生微服务体系共同构成了系统在“智能能力”与“工程稳定性”之间的关键桥梁。

LLM Agent 智能推理策略与 Prompt 工程实现

1. LLM Agent 服务定位与整体架构

在本项目中，出于工程上的可维护性与可控性考虑，我们将大语言模型相关的逻辑独立设计为一个单独的 Agent 服务（agent-service），而不是直接嵌入在后端主服务中。

在实际开发和调试过程中我们发现，大模型的调用具有明显的不稳定性特征：一方面其输出格式有时会不符合预期（例如 JSON 不闭合、字段缺失等），另一方面在内容层面也可能出现“看起来合理但实际不严谨”的幻觉问题。如果将这些不稳定逻辑直接掺杂在主业务服务中，会显著增加后端系统的复杂度与调试成本。

因此，我们将 Agent 设计为一个独立部署、独立容错、通过 HTTP 接口调用的微服务，其在整个系统中的定位可以概括为：

- 负责所有与 LLM 相关的推理与生成逻辑；
- 对外提供结构稳定、格式固定的知识图谱生成接口；
- 在内部通过多阶段生成与校验机制，尽可能降低大模型不稳定性对系统整体的影响。

从系统架构上看，整体调用关系如下：

- 前端 → 后端（backend）
- 后端 → agent-service（通过 /graph/build 接口）
- agent-service → LLM API（由内部 LLMClient 统一封装）

这样的拆分方式带来了几个直接好处：

- 解耦主业务与大模型调用逻辑：后端只关心“请求-结果”，不需要处理复杂的 Prompt 或异常情况。
- 便于独立部署与扩展：Agent 服务可以单独容器化部署，在需要时可以独立扩容或替

换模型实现。

- 便于集中处理不稳定性问题：所有与 JSON 修复、结果校验、降级兜底相关的逻辑，都集中在 Agent 内部实现，而不是分散在各个业务模块中。

从工程实现上看，agent-service 采用 FastAPI 构建，具备如下基础能力：

- 提供标准化的 HTTP 接口 (/graph/build 与 /healthz) ；
- 支持 CORS，便于前后端分离部署；
- 通过统一的请求与响应模型，保证即使在失败情况下也返回结构化 JSON，而不是不可控的异常页面；

内部封装 LLMClient，屏蔽不同模型接口的差异。

在云原生部署形态上，该 Agent 服务与后端服务一样，均以容器方式部署 (Docker)，属于系统中的一个独立微服务节点。在后续扩展中，该服务可以进一步接入缓存 (如 Redis) 或异步队列，以应对高并发或重复请求场景。

2. Agent 的多阶段推理链路设计 (Step0 ~ Step4)

在系统实现初期，我们曾尝试采用单次 Prompt 直接生成完整知识图谱（同时包含节点与关系）的方案。然而在实际测试过程中发现，该方式在工程稳定性与结果一致性方面均存在较大不确定性：一方面模型输出的结构化格式容易出现不完整或不符合约定 Schema 的情况，另一方面在语义层面也容易出现节点集合与关系集合不一致的问题，从而显著增加后处理的复杂度。

基于上述问题，本系统最终采用了一种多阶段、逐步约束的推理流水线结构。该结构的核心思想是将一次复杂的生成任务拆解为若干语义目标清晰、可分别校验的子步骤，并在每一阶段引入必要的工程约束，从而提高整体流程的可控性与稳定性。

整体流程在 agent-service/api.py 中被明确组织为 Step0 至 Step4 五个阶段，并由接口层统一调度执行。

2.1 Step0: 输入校验与语义规范化

Step0 的主要职责是对用户输入的概念进行可用性判断与基础规范化处理，以避免明显不适合构建知识图谱的输入进入后续流程。

该阶段主要完成以下工作：

- 对输入字符串进行基础清洗（去除多余空白、长度与编码合法性检查）；
- 结合 LLM 对输入概念的语义可建模性进行判断，例如：
 - 是否为具有明确语义边界的概念；
 - 是否适合作为知识图谱的中心节点展开；
- 在必要情况下，对概念名称进行一定程度的规范化处理。

若 Step0 判定输入不可用 (valid = false)，系统将直接返回 HTTP 400 错误，并附带失败原因说明，而不会继续调用后续任何生成步骤。该策略可以在流程最前端拦截高风险输入，从而减少无效的模型调用与无意义结果的产生。

2.2 Step1 与 Step2: 节点生成与语义信息补充

在节点构建阶段，系统并未尝试在单一步骤中生成包含完整语义信息的节点结构，而是将其拆分为两个阶段完成：

- Step1: 生成候选节点集合

- **Step2:** 为候选节点补充语义信息

在 **Step1** 中，系统仅要求模型围绕核心概念生成一组具有代表性的相关子概念或组成部分，用于构成图谱的节点集合。此阶段的目标是确定“图中应当包含哪些节点”，而不关注节点的详细描述或节点之间的关系结构。

在 **Step2** 中，系统在 **Step1** 生成的节点集合基础上，进一步请求模型为各节点补充必要的语义信息（例如简要定义、别名或其他说明性字段，具体以实际实现为准）。随后，通过合并逻辑对 **Step1** 与 **Step2** 的结果进行整合与去重，形成后续流程中唯一使用的节点集合 **merged_nodes**。

采用上述拆分策略的主要原因包括：

- 降低单次生成任务的复杂度，从而提高模型输出结构稳定性的概率；
- 避免在同一 **Prompt** 中同时要求模型完成概念构造与信息补全，导致输出不稳定；
- 为后续关系生成阶段提供一个语义边界清晰、数量可控的节点全集。

2.3 Step3: 在固定节点集合上的关系生成

在 **Step3** 中，系统基于已经确定的 **merged_nodes** 集合生成节点之间的关系结构 (**edges**)。

在该阶段，系统采用了一个重要的工程约束策略：关系的生成必须严格限定在既有节点集合范围之内，不允许引入新的节点实体。这一约束可以有效避免关系集合与节点集合之间出现引用不一致的问题，并显著降低后处理阶段的结构修复成本。

每一条关系边在结构上被要求包含以下字段：

- **source** 与 **target**
- **relation**（关系语义描述）
- **relation_type**（关系类型标识）
- **confidence**（置信度）
- **evidence**（简要证据说明）
- **direction**（方向性）

这些字段为后续的质量控制、置信度过滤与一致性校验提供了必要的信息基础。

2.4 Step4: 工程后处理与统一校验 (Check Layer)

在完成 **Step3** 的关系生成后，系统并不会直接将模型输出结果返回给上游服务，而是统一进入 **Step4** 进行工程层面的后处理与校验。

该阶段的核心目标是：将模型生成的、可能存在结构或质量问题的中间结果，转化为满足系统约束条件的最终可用图谱数据。

在 **Step4** 中，系统会对节点与边集合执行一系列确定性的规则处理，例如（具体规则以实现为准）：

- 节点与边数量的裁剪（如 **max_nodes** / **max_edges**）；
 - 基于置信度阈值的关系过滤；
 - **source** / **target** 与节点集合的一致性校验；
 - 去重与命名规范化处理；
 - 在 **meta** 信息中记录本次后处理所执行的主要裁剪与过滤操作说明。
- 只有通过 **Step4** 校验后的结果，才会被视为最终对外输出的知识图谱结构。

2.5 小结：多阶段推理结构的设计动机

综合来看，本系统采用多阶段推理流水线的主要目的在于：

- 将一次复杂且高不确定性的生成任务拆解为多个可控、可检查的子步骤；
- 通过阶段性约束逐步收敛模型的输出空间；
- 在最终输出前引入工程级校验层，以确保系统对外提供的数据结构与基本语义质量的稳定性。

该设计从工程角度将大语言模型视为一个能力强但不稳定的组件，并通过流程拆分与规则约束，将其纳入一个可控的系统框架之中。

3. Prompt 工程与 Agent 推理策略设计

为了提高跨学科知识图谱生成的稳定性和可靠性，我们采用了分阶段 Prompt 的设计方法。

3.1 设计思路：以“分阶段 Prompt”实现工程化 CoT

分阶段设计的核心思路是将复杂任务分解为多个语义清晰、可校验的子步骤。每个阶段都通过独立的 Prompt 明确输入输出要求，并在每个步骤中引入必要的工程约束，确保每个阶段的结果都可以被进一步验证和修正，从而提高整个过程的可控性和稳定性。

在我们的系统中，推理链条被划分为 Step0 至 Step4 五个阶段。每个阶段通过独立的 Prompt 与模型交互，最终通过 Step4 的工程后处理与统一校验确保最终结果的质量。这样分阶段的设计使得整体系统具有更高的灵活性和可操作性。

3.2 Step0: 概念校验与规范化 Prompt 设计

在整个 Agent 推理流水线中，Step0 并不承担任何内容生成任务，而是被设计为一个前置的输入质量控制与语义规范化模块。其核心目标是：判断用户输入是否具备作为“跨学科知识图谱构建中心概念”的语义合理性，并在通过校验的前提下，对概念名称进行统一规范化处理。

如果缺少该阶段的前置校验，系统在实际使用中容易面临以下问题：

- 用户可能输入生活化词语、情绪表达、完整句子甚至 URL 等非概念性文本；
- 对于这类输入，大模型往往仍会尝试“补全语义”并生成看似合理但缺乏学术意义的结构化结果；
- 这类偏差一旦进入后续节点生成与关系构建阶段，将导致整个图谱在语义层面失去严肃性，但在工程结构上却不易被直接发现。

基于上述考虑，系统在流水线最前端单独设置 Step0，用于完成概念可用性判定与名称规范化，将“该输入是否值得进入图谱构建流程”这一决策前移至流程起点，从源头降低错误语义扩散的风险。

3.2.1 Step0 Prompt 模板

（以下为实际使用 Prompt 的结构化模板示意，完整版本见代码仓库）

你是“概念校验与规范化（Step0）”模块。

输入概念：{concept}

任务：

判断该输入是否是一个适合用于构建“跨学科知识图谱”的【抽象 / 学术概念】。

若 valid=true，则输出规范化中文名、可选英文名与少量别名；

若 `valid=false`，则输出空概念并给出原因；
输出必须为严格 JSON，禁止任何额外说明文本。

输出格式：

```
{
  "valid": true/false,
  "normalized_concept": "...",
  "canonical_en": "...",
  "aliases": ["...", "..."],
  "reason": "..."
}
```

关键约束：

若不确定，宁可判定为 `valid=false`

`normalized_concept` 长度受限

`aliases` 数量受限

`reason` 仅允许一句话说明

该 Prompt 的完整内容在工程中以模板文件形式维护，并通过参数注入的方式由 Step0 模块动态调用。

3.2.2 Prompt 的结构化约束设计

从设计上看，Step0 并不是一个开放式问答 Prompt，而是一个高度结构化的“判定 + 抽取”型任务描述，其约束主要体现在以下几个方面：

(1) 角色约束

通过明确指定模型扮演“概念校验与规范化模块”，将其行为限定在判定与抽取任务范围内，而非自由语义扩展。

(2) 任务流程约束

将任务拆解为两个阶段性目标：

- 首先判断 `valid / invalid`；
- 仅在 `valid=true` 的前提下才允许输出规范化概念信息。

(3) 输出结构强约束

- 仅允许输出 JSON 对象；
- 字段结构固定；
- 明确禁止输出任何额外解释性文本。

(4) 字段级规模约束

- 限制 `normalized_concept` 的长度；
- 限制 `aliases` 的数量；
- 限制 `reason` 的表达形式为单句说明。

(5) 风险控制策略约束

通过明确加入“若不确定，宁可判定为 `invalid`”的指令，在策略层面引导模型采取保守决策，从而减少语义边界模糊概念被误放行的概率。

3.2.3 Step0 在整体智能逻辑中的功能定位

从系统整体结构来看，Step0 主要承担以下几项功能：

(1) 输入语义门禁 (Semantic Gate)

在流水线最前端拦截明显不适合进入知识图谱构建流程的输入类型。

(2) 概念命名规范化 (Normalization)

将用户输入的概念名称统一为后续步骤中使用的标准化锚点。

(3) 降低幻觉扩散风险

避免模型在明显不适合建模的输入上构造“形式正确但语义空洞”的知识结构。

(4) 为后续推理提供稳定中心节点

Step1~Step3 的节点生成与关系构建均严格围绕 Step0 输出的 `normalized_concept` 展开。

在工程实现中，一旦 Step0 返回 `valid=false`，系统将直接以 HTTP 400 终止请求流程，不再调用后续任何生成步骤，从而在计算成本控制与语义质量控制两个层面同时实现前置约束。

3.2.4 体现的工程化智能设计思想

Step0 的设计反映了本系统在整体架构上的一个基本原则：

不将大语言模型视为“天然可靠的智能决策体”，而是将其视为一个能力强但不稳定的语义组件，必须通过明确的角色设定、任务拆解与结构化约束，纳入可控的工程流程之中。

Step0 的目的并非提升模型的“创造性”，而是通过限制其在错误问题空间中的自由度，为整个 Agent 系统建立一个语义稳定的入口，从而提高后续多阶段推理结果的整体可靠性与一致性。

3.3 Step1: 候选节点生成 Prompt 设计 (跨学科节点构建)

在完成 Step0 的输入校验与概念规范化之后，系统进入知识图谱构建流程的第一个生成阶段：围绕核心概念构造一组跨学科候选节点集合。需要强调的是，Step1 的目标并非直接生成完整的图结构，而是首先确定“图谱中应当包含哪些关键概念节点”，从而为后续的语义补充与关系构建阶段提供一个稳定、可控的概念全集。

在设计上，本系统并未采用开放式的“相关概念自由联想”策略，而是通过一个强约束、结构化、多目标覆盖的 Prompt，将节点生成过程转化为一个可被工程化控制的构造任务。该阶段的核心约束包括：

- 必须覆盖预设的五个学科域（数学、物理、计算机/AI、生物/神经科学、社会科学/经济学）；
- 每个学科域至少生成一个节点；
- 明确区分紧密相关、一般相关与弱相关但合理的节点；
- 对弱关联节点显式降低 `confidence`，并在 `summary` 中说明其关联性质；
- 严格限定输出为结构化 JSON，仅包含 `nodes`，不允许输出任何解释性文本。

通过上述约束，Step1 被定位为一个受控的跨学科节点空间构建阶段，而非简单的概念枚举过程。

3.3.1 Step1 Prompt 模板

(以下为结构化摘要示意，完整版本见工程仓库)

你是“跨学科知识图谱生成 (Step1: 候选节点)”模块。

输入：

```
normalized_concept: {normalized_concept}  
canonical_en: {canonical_en}  
aliases: {aliases_json}
```

系统固定学科域:

["数学", "物理", "计算机/AI", "生物/神经科学", "社会科学/经济学"]

任务:

围绕核心概念, 在每个学科域内生成相关概念节点, 并输出统一的 **nodes** 列表。

输出:

只能输出 JSON

只能包含 "nodes" 字段

每个节点必须符合规定字段结构与数量分布要求

在实际实现中, 该 Prompt 对节点字段结构、数量分布、质量要求与 JSON 格式约束均进行了完整定义, 这里不再逐条展开。

3.3.2 设计目标: 受控的跨学科覆盖式节点构造

Step1 的核心设计思想, 并非简单询问“有哪些相关概念”, 而是要求模型完成一个带有明确覆盖目标与规模约束的构造任务, 其约束体系主要体现在以下几个方面:

(1) 覆盖约束

- 五个学科域必须全部出现;
- 每个学科域至少生成一个节点。

(2) 规模约束

- 核心节点固定为 1 个;
- 每个学科域 1–3 个节点;
- 总节点数建议控制在 11–13 个范围内。

(3) 角色类型区分

- 明确区分 core、bridge 与 normal 三类节点, 以支持后续跨域关系建模。

(4) 相关性分级建模

- 紧密相关节点: importance 较高 (≥ 0.7);
- 一般相关节点: importance 处于中等区间 (0.5–0.8);
- 弱相关节点: importance 较低, 且 confidence ≤ 0.55 。

通过上述结构性约束, 可以避免模型仅在其“最熟悉的领域”内过度展开, 从生成目标层面强制引入跨学科视角。

3.3.3 节点字段的 Schema 约束设计

在 Step1 中, Prompt 对每一个节点的输出结构规定了严格的 Schema 约束, 主要包括以下字段:

- id:
 - 仅允许包含 [A-Za-z0-9_-];
 - 不可重复;
 - 核心节点固定使用 {core_id}。
- label:

以中文为主，可包含常用英文缩写。

- **field:**
必须属于以下枚举集合之一：
"核心", "数学", "物理", "计算机/AI", "生物/神经科学", "社会科学/经济学"
- **type:**
仅允许 "core" / "bridge" / "normal"。
- **summary:**
不超过 180 字，单段；
必须同时说明：
① 该概念的基本含义；
② 与核心概念的关系；
③ 在本学科中的作用。
- **aliases:**
字符串数组，可为空。
- **importance:**
0.0 – 1.0，表示该概念在所属学科中的重要性。
- **confidence:**
0.0 – 1.0，表示与核心概念关联的确信度；
对弱关联节点明确要求 ≤ 0.55 。

上述字段约束的工程意义在于：将“概念列表”提升为带有语义说明、重要性权重与确信度标注的结构化节点集合，并为 Step4 的工程裁剪（importance 排序、置信度过滤等）提供直接可用的数值依据。

3.3.4 弱关联节点的显式建模策略

在跨学科场景中，并非所有学科都与核心概念存在同等强度的语义耦合。为避免模型在弱相关领域中强行编造牵强关系或完全忽略某些学科域，系统在 Prompt 中显式引入“弱关联但合理”节点类型，并施加以下约束：

- 弱关联仅允许基于：
方法论映射（如不确定性度量、统计描述、优化目标等）；
典型应用场景；
可解释的类比关系。
- **summary** 中必须明确说明其关联性质；
- **confidence** 必须 ≤ 0.55 。

该策略的目的在于：将“跨学科覆盖”这一结构性需求，显式建模为带不确定性标注的信息结构，而不是在语义层面人为拉近原本关联度有限的概念。

3.3.5 Bridge 节点的结构性引导设计

Prompt 中进一步引入 bridge 类型节点，用于标记：

- 在多个学科中均具有意义；
- 或在跨学科解释中具有中介作用的概念。
并通过以下规则进行约束：
- 每个学科域最多 0 – 1 个 bridge 节点；
- bridge 节点仍必须隶属于某一个具体学科域，而不是引入虚拟的“跨域类别”。
这一设计为后续 Step3 的跨域关系生成，以及 Step4 中“亮点边优先保留”策略提

供了明确的结构基础。

3.3.6 输出格式约束与反幻觉控制

Step1 对输出格式施加了严格的工程约束：

- 只能输出一个 JSON 对象；
- 只能包含 "nodes" 字段；
- 禁止输出 edges；
- 禁止任何解释性文本、Markdown 或代码块标记；
- 输出必须可被 json.loads() 直接解析；
- 强制使用英文双引号，禁止逗号。

在语义质量层面，Prompt 进一步明确要求：

- 所有节点必须是真实存在、可验证的概念；
- 禁止捏造不常见专有名词、论文标题或定律名称。

这些约束从结构层与语义层两个维度同时压缩模型的幻觉空间。

3.3.7 Step1 在整体 Agent 推理链路中的功能定位

综合来看，Step1 在整体系统中主要承担以下职责：

- 构建一个跨学科覆盖、规模受控、带质量标注的节点全集；
- 将知识图谱构建流程从一开始限定在一个有限、可解释、可校验的概念空间内；
- 为后续 Step2 的节点语义增强，Step3 的受限关系生成，Step4 的工程级质量控制，提供统一且稳定的结构基础。

从 Agent 系统设计角度看，Step1 并非“让模型列举若干相关术语”，而是通过 Prompt 将模型纳入一个受 Schema、数量分布与跨学科目标多重约束的结构化构造过程之中。

3.4 Step2: 节点语义补充 Prompt 设计 (语义特征工程)

在 Step1 中，系统已经构建了一组覆盖多个学科域的候选节点集合。然而，此时的节点仍主要承担“结构占位”的作用，其语义信息虽然包含 summary、importance、confidence 等字段，但在粒度与结构上仍不足以直接支撑后续两个关键任务：

- Step3 中对节点之间关系类型的判断与 evidence 的生成；
- Step4 中对跨域关系、类比关系可解释性与弱关联节点的规则化过滤。

为此，系统在进入关系生成之前，专门引入 Step2，作为一个语义补充与结构化摘要阶段，用于将每个节点转化为一个“可被后续步骤直接消费的语义特征单元”。

从工程角度看，Step2 并非简单的文本扩写，而是承担了一种轻量级语义特征工程 (Semantic Feature Engineering) 的角色。

3.4.1 Step2 Prompt 模板

(以下为结构化摘要示意，完整版本见工程仓库)

你是“跨学科知识图谱生成 (Step2: 节点语义补充)”模块。

输入：

一个 nodes 数组 (包含 id, label, field, type, summary, aliases, importance, confidence) 。

目标:

为每个节点补充用于后续关系生成与规则过滤的短语义字段。

输出:

只能输出 JSON;

只能包含 "nodes";

每个元素仅包含: id, details, key_points, reading_hint;

必须覆盖所有输入节点, 不得新增、删除或改写 id。

在实际工程中, 该 Prompt 对字段长度、数量、格式与解析安全性均施加了完整约束, 这里不再逐条展开。

3.4.2 设计目标: 将“概念节点”转化为“可计算的语义特征单元”

Step2 的核心设计目标, 并不是再解释一遍这些概念, 而是将每一个节点转换为一个结构化、粒度统一、可被后续步骤直接使用的语义特征载体。

具体而言, Step2 为每个节点补充且仅补充以下四个字段:

- **id:**
用于与 Step1 中的节点一一对应 (不允许新增、删除或改写)。
 - **details:**
中等长度的概念说明 (约 80 – 160 字), 用于:
 - 支撑关系语义判断;
 - 作为 evidence 生成时的语义素材来源。
 - **key_points:**
固定 3 条的要点短语, 用于提供高度压缩的语义标签; 在规则判断与跨节点对比中作为“离散语义特征”。
 - **reading_hint:**
一句简短的使用提示或易混点说明, 用于提升人类可读性, 同时作为辅助模型区分相似概念的弱监督信号。
- 通过这一设计, 节点在进入 Step3 之前, 已经被转化为带有多粒度语义表征的结构化单元。

3.4.3 结构化输出与长度控制约束

Step2 的 Prompt 对输出格式与内容长度施加了严格的工程级约束, 以保证结果的可解析性与可用性:

(1) 结构约束

- 只能输出一个 JSON 对象;
- 只能包含 "nodes" 数组;
- 每个元素只能包含: id, details, key_points, reading_hint;
- 必须覆盖输入中的每一个 id, 不得新增、删除或改写;
- 输出顺序尽量与输入保持一致。

(2) 解析安全约束

- 禁止输出任何 Markdown、解释性文字或代码块标记;
- 禁止使用单引号;
- 禁止尾逗号;

- 禁止输出 `null`;
- 明确要求可被 `json.loads()` 直接解析。

(3) 内容长度与格式约束

- **details:**
单段中文，不允许换行；
长度 80 – 160 字（上限 180）；
必须同时包含：
① 是什么；② 与所属学科或核心概念的关系；③ 一个典型用途或作用。
- **key_points:**
固定 3 条；
每条不超过 20 字。
- **reading_hint:**
1 句；
不超过 25 字；
用于表达“常见误区 / 使用提醒 / 概念区分”之一。

这些约束的工程目的在于：将模型输出压缩为结构稳定、粒度一致、可被程序可靠消费的语义摘要形态，避免生成冗长、松散且难以复用的自由文本描述。

3.4.4 与 Step3 / Step4 的协同关系

Step2 的输出在系统中具有明确的“中间语义表示（Intermediate Semantic Representation）”定位：

对 Step3:

- **details** 与 **key_points** 可作为关系判断与 **evidence** 生成的主要语义依据；
- **reading_hint** 可在相似概念区分时提供补充信号。

对 Step4:

- 可用于判断某些关系是否具备“最低限度的解释性”；
- 也可辅助识别弱关联节点与跨域关系是否具有基本合理性。

从工程视角看，Step2 实际上承担的是为后续推理步骤构造稳定语义输入空间的角色。

3.4.5 Step2 在整体 Agent 推理策略中的意义

将 Step2 独立设置为一个单独阶段，而不是与 Step1 或 Step3 合并，其主要工程动机包括：

- 降低单个 Prompt 的任务复杂度，提高模型输出的结构稳定性；
- 在节点层面提前构建统一风格、统一粒度的语义描述，为后续关系层推理提供一致的参照系。

这使得整个 Agent 的推理流程呈现出清晰的分层结构：

结构骨架构建（Step1）

→ 节点语义特征工程（Step2）

→ 受限关系生成（Step3）

→ 工程校验与收敛（Step4）

而不是在单一步骤中要求模型同时完成多种异构任务，从而显著提升了整体流程的可控性与稳定性。

3.5 Step3: 关系边生成 Prompt 设计（受限连边与可解释推理）

在 Step1 与 Step2 完成后，系统已经获得一组固定的节点集合，并为每个节点补充了可供理解与推理使用的结构化语义信息。Step3 的任务是在此基础上生成知识图谱的关系边（edges），并在满足结构稳定性的前提下，尽可能体现跨学科联结与关系可解释性。

从工程实践角度看，关系生成阶段是整个流水线中幻觉风险最高的环节之一：模型可能引入不存在的实体、生成难以验证的关系类型，或输出不符合约定 Schema 的结构化结果。基于此，Step3 并未被设计为一个开放式关系生成过程，而是通过强约束 Prompt 将连边任务限定在一个可控、可校验、可后处理的搜索空间内，并将“输出结构可解析性”与“节点引用一致性”作为最高优先级目标。

3.5.1 Step3 Prompt 模板

（以下为核心约束摘要，完整版本见工程仓库）

仅允许输出一个 JSON 对象：

```
{"edges":[...]}
```

每条边必须符合固定字段结构与枚举约束：

source / target 必须从 nodes_json 中复制已有 id，不允许新增或改写；

目标生成 12 – 18 条边，并满足跨域覆盖与可解释性要求（reasoning_chain）。

3.5.2 关键约束一：将连边空间限定在“固定节点集合”内

Step3 Prompt 中引入的最核心约束之一是：

source 与 target 只能从输入的 nodes_json 中已有 id 复制，不允许改写、翻译或新增任何节点标识。

该约束从机制上禁止模型在关系生成阶段引入新实体，从而有效避免以下常见问题：

- 生成引用不存在节点的“悬挂边”；
- 出现同一概念被重复命名为不同 id 的现象；
- 为了“组织关系结构”而临时虚构桥梁概念。

这一“先固定节点集合，再进行受限连边”的策略，与 Step4 的一致性校验机制形成互补：

- Prompt 层负责事前压缩搜索空间；
- Check Layer 负责事后进行规则过滤与质量收敛。

3.5.3 关键约束二：严格的 Edge Schema 与枚举控制

Step3 对每一条关系边规定了固定的 7 个必填字段：

- source / target: 节点 id
- relation: 中文短语（≤ 16 字）
- relation_type: 必须来自固定枚举集合
- confidence: 数值型浮点数（不得加引号）
- evidence: 中文一句话说明（≤ 35 – 40 字）
- direction: directed / undirected

其中，relation_type 被限制在以下枚举集合中：

直接关联 / 类比关联 / 数学关联 / 先修依赖 / 应用关联 / 因果/机制 / 相对/对比 /

其他

该类枚举约束在工程上具有两方面意义：

- 抑制语义漂移：避免模型自行发明关系类别，导致后处理无法归类；
- 便于规则治理：为 Step4 中“类比关系解释性要求”“跨域亮点边统计”等规则提供稳定的类型依据。

3.5.4 关键约束三：输出可解析性优先的工程策略

Step3 Prompt 对 JSON 输出格式施加了极为严格的结构约束，包括但不限于：

- 必须以 { 开头，以 } 结尾；
- 所有字符串必须使用英文双引号；
- 禁止尾逗号；
- 禁止插入任何解释性文本；
- 若无法同时满足数量或覆盖要求，应优先减少 edges 数量以保证 JSON 完整性。

该策略的工程目标在于将“输出结构可解析、服务不崩溃”作为第一优先级，避免因格式错误直接导致整个生成流程失败。同时，这也为 Step4 的后处理提供了必要前提：只有在结构可解析的情况下，后续的规则过滤、一致性校验与质量控制才具有实际意义。

3.5.5 跨域覆盖与图结构目标约束

为了避免生成结果退化为“围绕核心概念的同域放射状结构”，Step3 在 Prompt 层面引入了明确的结构性目标约束：

- 核心连接覆盖：
若节点集中包含五个学科域，则每个学科域至少生成一条与核心概念相连的边；
- 跨域横向连接：
至少生成 4 条满足 $source.field \neq target.field$ 的跨学科边；
- 边去重约束：
(source, target, relation_type) 组合不得重复；
- 方向性约定：
因果 / 依赖类关系 → directed
类比 / 对比类关系 → undirected

这些约束使“跨学科”不再停留在节点集合层面，而是在图结构层面体现为真实存在的横向连接关系。

3.5.6 可解释性机制：reasoning_chain 与 bridge_concepts

为满足对“推理链路 (CoT)”与“事实支撑”的要求，Step3 在结构层面引入了两个可选字段，并对其使用场景进行明确限定：

- reasoning_chain (可选) :
由 2 – 4 个短步骤组成；
至少 2 条跨域边必须包含该字段；
用于说明跨域连接的逻辑路径，降低“空洞类比”的风险。
- bridge_concepts (可选) :
若出现，必须为 nodes 中已存在的 id；
用于显式标记跨域连边所依赖的中介概念 (与 Step1 中的 bridge 节点设计相呼应)。
需要强调的是：本系统中的“推理链路”并非要求模型输出完整思考过程，而是通过分

阶段推理结构 (Step0 ~ Step3)，以及关键跨域边上的最小解释链 (reasoning_chain)，构建一种工程化、可审阅的弱 CoT 机制，使跨域关系不仅可以生成，而且可以被人类理解与检查。

3.5.7 Step3 在整体反幻觉体系中的位置

在整体架构中，Step3 的 Prompt 约束与 Step4 的 Check Layer 共同构成一套“事前约束 + 事后治理”的反幻觉机制：

Step3（事前约束）：

- 限定节点引用范围；
- 限定关系类型枚举；
- 要求 evidence 与可选 reasoning_chain。

Step4（事后治理）：

- 过滤低置信度与缺乏解释性的边；
- 在跨域覆盖不足时进行“救图式”降阈值调整；
- 执行亮点优先裁剪与一致性校验。

通过这一组合机制，系统不仅能够生成关系边，而且能够在工程层面对关系质量进行持续约束与修正，使最终输出更符合课程对事实准确性、Prompt 严谨性与异常可处理能力的要求。

3.6 Agent 编排与工程化推理链路 (Orchestration as CoT)

本系统的 LLM Agent 未采用单次 Prompt 直接生成完整图谱 (nodes+edges) 的方案，而是将图谱构建拆解为 Step0→Step1→Step2→Step3 的分阶段流水线，并在末端设置 Step4 作为工程校验与结果收敛层。该编排方式体现为一种工程化的推理链路：系统不要求模型输出冗长的显式思考过程，而是通过任务拆分、输入边界控制、统一 Schema 与阶段性校验，使输出逐步收敛到结构可解析、语义可控的图谱结果。

在实现上，链路由 agent-service/api.py 统一调度，核心逻辑集中在 agent_pipeline.py。接口接收概念词后首先执行 Step0，对输入进行可用性判定与规范化：结合规则过滤（如长度、URL/疑问句等）与 LLM 的结构化判定，输出 valid、normalized_concept、aliases 等结果；若判定不可用，则流程直接终止，避免无效调用及由错误输入引发的语义偏差向后传播。

Step0 通过后进入 Step1，用于生成候选节点集合。该阶段将节点生成约束为“跨学科覆盖”的结构化抽取任务：围绕规范化后的核心概念，在固定的学科域范围内输出节点，并给出 summary、importance、confidence 等字段，为后续治理提供可操作的数值依据。工程实现中，Step1 对结构化输出稳定性采取了更稳健的处理策略：优先严格解析 JSON；解析失败时进行保守的本地修复；必要时触发二次 LLM 修复以提高可解析率，从而降低因结构化输出不稳定导致的链路中断。

Step2 在不改变 Step1 节点集合的前提下补充节点语义信息，将节点从“概念占位”提升为可供关系推断与质量过滤使用的中间语义表示。为控制 Prompt 长度与降低截断风险，Step2 会对输入节点字段进行裁剪，仅保留关系生成所需的关键字段，并要求按固定 Schema 输出 details、key_points、reading_hint。完成后，系统执行覆盖性校验，确保每个输入 id 都获得对应的语义补充，避免后续连边阶段出现引用与语义信息不一致的问题。随后通过 merge_step2_into_nodes 将 Step2 的语义字段合并回节点结构，形成后续唯一使用的 merged_nodes。

Step3 在固定节点集合上生成关系边 (edges)。该阶段优先保证节点引用一致性与输

出可解析性，并在此基础上满足跨域连接与可解释性的结构目标。实现中，Step3 同样对输入节点进行字段压缩以降低负载，并对生成结果执行 schema 清洗与强约束过滤：校验 source/target 引用合法性、relation_type 与 direction 的枚举合法性、confidence 的数值可解析性，并对 relation 与 evidence 进行必要的规范化与裁剪。对于 reasoning_chain、bridge_concepts 等可选字段，仅在满足步数与引用合法性时保留；当有效边数量不足时，链路会触发一次更强约束的重试策略，以提升输出的可用性与一致性。

最终，nodes 与 edges 进入 Step4 (step4_postprocess_graph) 进行工程级质量控制与统一收敛。Step4 作为 Check Layer，以确定性规则对模型输出进行治理：字段合法性校验、去重、解释性要求约束、置信度阈值过滤，以及在跨域覆盖不足时的阈值调整与亮点优先裁剪等。处理结果会在 meta.notes 中记录关键治理信息，便于调试与审阅。

综上，系统将大模型置于可控的工程流水线之内：各阶段均具备明确的输入边界、输出 Schema 与可验证的约束条件，并通过阶段性校验与末端 Check Layer 形成闭环，从而实现可追踪、可治理、可复现的推理链路。

4. 容错机制与反幻觉设计 (Step4 Check Layer)

LLM 生成的图谱结果在结构与内容层面都具有不确定性：常见问题包括字段缺失、枚举值不合法、证据文本空洞、节点与边引用不一致、边数量失控以及置信度标注失真等。为降低上述风险，本系统在 Agent 流水线末端引入 Step4 (step4_postprocess_graph) 作为工程保证层 (Check Layer)，对 Step3 的模型输出执行确定性校验、清洗、过滤与降级策略，确保最终对外输出的 GraphJSON 满足结构稳定、规模可控、质量可解释的要求。

Step4 的整体目标可以概括为三点：

- 结构安全：输出必须满足 GraphJSON 约定，字段合法、引用一致；
- 质量可控：通过置信度阈值、解释性约束与去重策略抑制幻觉边；
- 图可用性：在保证质量的前提下维持跨域覆盖与跨域连接数量，避免“只剩单域碎片图”。

4.1 配置化质量门槛与规模约束 (DEFAULT_STEP4_CFG)

Step4 将关键策略参数集中抽象为配置项 (DEFAULT_STEP4_CFG)，并支持通过接口 cfg 进行覆盖，以便在不同演示场景或不同课程指标下调整约束强度。主要配置项包括：

- 规模控制
 - max_nodes: 最多保留节点数 (默认 20)
 - max_edges: 最多保留边数 (默认 30)
- 置信度阈值
 - min_edge_confidence: 边的基础保留阈值 (默认 0.55)
 - min_edge_confidence_floor: 当跨域/覆盖不足时，允许自动降低到的下限 (默认 0.35)
- 跨域与覆盖目标
 - min_domains_covered: 要求最终图至少覆盖的学科域数量 (默认 4)
 - min_cross_domain_edges_all: 跨域边最低数量 (按 field 不同计算)
 - min_cross_domain_edges_highlight: 跨域“亮点边”最低数量 (还要求 relation_type 在指定集合中)
 - cross_domain_mode: 跨域标记口径 (all/highlight)
- 解释性要求 (抑制空洞推理)
 - require_reasoning_for_types: 某些关系类型必须提供 reasoning_chain / sources / bridge_concepts 之一 (默认对“类比关联”启用)
 - allow_evidence_only_for_reasoning_required: 是否允许仅靠较长 evidence 放行 (默

认 False, 更严格)

- 枚举白名单

allowed_fields: 节点学科域枚举 (包含“核心、数学、物理、计算机/AI、生物/神经科学、社会科学/经济学”)

allowed_relation_types: 关系类型枚举 (直接关联、类比关联、数学关联、先修依赖、应用关联、因果/机制、相对/对比、其他)

这样做的意义在于: **Step4** 的行为是可解释且可调参的, 不依赖隐式经验规则; 也便于在文档中将“质量控制策略”以清单形式明确呈现。

4.2 节点侧校验: 域枚举合法性 + 去重保优

Step4 首先对节点集合执行基础一致性检查, 避免后续边校验失效。

(1) 节点基础合法性过滤

对每个 **node** 执行:

- 必须存在 **id**, 否则剔除 (计数 **node_missing_id**)
- **field** 必须在 **allowed_fields** 枚举集合中, 否则剔除 (计数 **node_bad_field**)

该策略用于避免模型输出出现:

- 空 **id** 或不可引用节点;
- 学科域乱填导致统计与跨域策略失真。

(2) 节点去重: 保留 **importance** 更高版本

当出现相同 **id** 的重复节点时:

- 比较 **importance**, 保留更高者
- 其余视为重复剔除 (计数 **node_duplicate**)

该策略保证节点集合在语义上“以更重要版本为准”, 同时为后续节点裁剪提供稳定排序依据。

4.3 边侧校验: 结构合法性 + 字段清洗 + 去重保优

边集合是幻觉风险最高的部分, 因此 **Step4** 对边执行更严格的结构化校验与规范化清洗。

(1) 边字段规范化 (Normalization)

对每条边执行 **_normalize_edge_fields_inplace**:

- **relation**、**evidence** 统一清洗为单行文本 (去换行、压缩空格)
- **relation** 长度裁剪: 同步 **Step3** 的上限 (默认 16, 历史兜底 32)
- **evidence** 长度裁剪: 最大 120
- **reasoning_chain**: 每步清洗并限制单步长度、最多保留 4 步
- **bridge_concepts** 清理空值
- **confidence** 强制转 float
- 可选: **drop_unknown_edge_fields=True** 时剥离非白名单字段 (提升输出安全性与一致性)

(2) 结构合法性校验 (Schema & Consistency)

边必须满足 **_edge_valid_structure** 的全部约束, 否则剔除 (计数 **edge_invalid_structure**):

- **source** / **target** 必须存在且不同
- **source/target** 必须引用存在的 **node id** (防止悬挂边)
- **relation_type** 必须在白名单枚举中

- `relation` 非空
- `evidence` 非空且清洗后长度 $\geq$ `min_evidence_len` (默认 8)
- `direction` 必须为 `directed` 或 `undirected`
- `confidence` 必须为正数 (≤ 0 直接剔除)

此外, 若检测到 `reasoning_chain` 存在但步骤数少于 `min_reasoning_steps` (默认 2), 系统将其视为不满足最小解释性要求并移除该字段, 以避免低质量推理链对后续校验与展示产生干扰。

(3) 边去重: 按置信度保留更优

边去重键可配置为:

- `coarse`: (`source`, `target`, `relation_type`) (默认)
- `fine`: (`source`, `target`, `relation_type`, `relation`)

发生重复时保留 `confidence` 更高的边, 其余剔除 (计数 `edge_duplicate`)。该策略用于控制关系爆炸与重复表达, 提高最终图的可读性。

4.4 解释性规则: 对高风险关系类型施加强约束

考虑到某些关系类型更容易产生空洞或幻觉 (典型为“类比关联”), `Step4` 引入解释性门槛:

若 `relation_type` $\in$ `require_reasoning_for_types` (默认包含“类比关联”), 则该边必须至少满足以下之一:

- `reasoning_chain`: 且步数 $\geq$ `min_reasoning_steps`
- `sources`: 非空列表
- `bridge_concepts`: 非空列表

否则剔除 (计数 `edge_missing_reasoning`)。

并且默认不允许仅凭 `evidence` 较长就放行 (`allow_evidence_only_for_reasoning_required=False`), 以避免出现“看似解释、实际空泛”的类比边进入最终图谱。

4.5 自动标注 (Tagging): 为后续裁剪与展示提供信号

`Step4` 会对保留下来的边自动打标签, 用于后续裁剪时的优先级控制以及前端展示时的高亮/分类 (如跨域连接、核心相关等)。

标签规则包括:

① 跨域连接:

- `cross_domain_mode="all"`: 只要 `source.field != target.field` 即标记
- `cross_domain_mode="highlight"`: 仅当关系类型属于 `cross_domain_relation_types` 才标记

② 无向关系: `direction == "undirected"` 标记“无向关系”

③ 核心相关: 若 `source/target` 命中核心节点 (`core_id`), 标记“核心相关”

核心节点的识别优先规则为:

- 若存在 `type == "core"` 的节点, 取其 `id`
- 否则取 `importance` 最大的节点作为核心节点 (兜底)

4.6 置信度过滤与“救图”降阈值策略 (兼顾质量与可用性)

(1) 基础阈值过滤

系统先按 `min_edge_confidence` (默认 0.55) 过滤边。

但为了保留结构性价值较高的连接, 以下类别可放宽阈值, 即使置信度略低也允许保留:

- 远亲概念 (`tag = keep_far_edge_tag`)
- 桥梁链路 (`tag = keep_bridge_tag`)
- 跨域连接 (`tag = “跨域连接”`)

这反映了一个工程权衡: 在跨学科图谱中, 桥梁与跨域边往往更稀缺, 也更能体现系统价值, 因此在质量过滤时会被优先保护。

(2) 覆盖/跨域不足时的自动降阈值

仅靠高阈值过滤有时会导致最终图过于“单域化”或边数过少。为避免出现不可用或缺乏跨域特点的结果, **Step4** 引入自动降阈值机制:

统计指标按“最终边诱导出的节点子图”计算 (更贴近最终可视化效果):

- `num_domains_covered`
- `num_cross_domain_edges_all`
- `num_cross_domain_edges_highlight`

若任一指标不足 (域覆盖不足或跨域边不足), 系统会:

- 从当前阈值开始, 每次降低 0.05, 直到达到 `min_edge_confidence_floor` (默认 0.35) 为止
- 在每个阈值下重新筛选 `edges`, 并重新计算覆盖与跨域指标
- 选择“更满足约束”的候选结果 (评分优先级: 域覆盖 > `cross_all` > `cross_highlight` > 图更简洁)

当找到满足约束的阈值时提前停止, 并在 `meta.notes` 中记录本次自动调整行为。

该策略的设计意图是: 在保证基本质量门槛的同时, 维持图谱的跨学科展示效果, 避免“过滤后只剩少量同领域边”的情况。

4.7 裁剪策略: 亮点优先, 保持图结构信息密度

当边数超过上限 (`max_edges`) 时, **Step4** 采用分桶优先策略 `_prune_edges_keep_highlights` 进行裁剪:

优先级从高到低为:

- ① 远亲概念 (`far`)
- ② 桥梁链路 (`bridge`)
- ③ 跨域亮点边 (`cross-highlight`)
- ④ 跨域边 (`cross-all`)
- ⑤ 普通边 (`normal`)

每个桶内部按 `confidence` 降序排列, 并在达到 `max_edges` 后截断。

当节点数超过上限 (`max_nodes`) 时, 采用 `_prune_nodes_by_importance`:

- 优先保留“被最终 `edges` 引用到的节点”以及核心节点
- 其余节点按 `importance` 降序补齐
- 节点裁剪后会同步移除引用到被裁剪节点的边, 保证一致性

该裁剪策略确保最终图在规模受控的前提下, 仍尽量保留对“跨域展示价值”更高的边与关键节点。

4.8 输出统计与可解释日志 (`meta.stats` 与 `meta.notes`)

Step4 在输出的 `meta` 中补充统计信息, 便于前端展示与调试:

- `num_nodes`

- num_edges
- num_domains_covered
- num_cross_domain_edges (兼容旧字段, 默认 all)
- num_cross_domain_edges_all
- num_cross_domain_edges_highlight

同时 meta.notes 会拼接 Step4 的执行摘要, 包括:

- 本次 Step4 耗时
- 各类剔除计数 (例如 edge_invalid_structure、edge_duplicate 等)
- 是否触发了自动降阈值救图
- 是否触发了 edges/nodes 裁剪

这种“统计 + 说明”的输出方式, 使系统在出现质量问题时能够快速定位原因, 并为课程验收提供可解释依据。

4.9 小结: Step4 作为系统级 Check Layer 的作用

Step4 将大模型生成结果纳入一组确定性工程规则之下, 主要解决三类风险:

- 结构风险: 保证 schema 合法、引用一致、枚举值可控;
- 幻觉风险: 通过解释性门槛、置信度过滤与去重抑制空洞关系;
- 可用性风险: 通过跨域/覆盖约束与“救图”机制, 避免输出退化为单域碎片。

这使得 Agent 输出不仅能“生成”, 更能在工程层面对不确定性进行约束与治理, 从而满足课程对智能逻辑严谨性与容错性的要求。

结论与工程经验总结

本文档围绕“跨学科知识图谱智能体”系统, 从后端架构设计、云原生部署形态、LLM Agent 工具链基础设施, 以及多阶段智能推理策略与 Prompt 工程实现四个层面, 系统性地阐述了一个面向结构化知识生成任务的工程级智能系统的完整设计与实现路径。

与将大语言模型简单作为“问答接口”或“内容生成工具”的用法不同, 本系统从一开始即将 LLM 视为一个能力强但不稳定的组件, 并通过微服务解耦、多阶段 Agent 编排、输出治理工具链与工程化 Check Layer, 将其纳入一个可约束、可校验、可回滚、可演化的系统框架之中。通过 Step0~Step4 的分阶段推理流水线, 系统将一次高不确定性的生成任务拆解为若干语义目标清晰、结构边界明确的子任务, 使推理过程在工程上逐步收敛, 而不是一次性押注于模型的“完整正确输出”。

在工程架构层面, 系统采用基于 FastAPI 的微服务设计, 并通过 Docker Compose 实现后端服务、Agent 服务、Neo4j 图数据库与 Redis 缓存的云原生编排部署, 体现了云原生应用在解耦、弹性、可观测性与可扩展性方面的工程优势。在此基础上, Agent 服务内部进一步构建了围绕 LLM 的“输出治理工具链”, 通过 JSON 修复、Schema 校验、字段白名单清洗与一致性检查等机制, 将模型输出从“不可靠文本”转化为“可被程序消费的结构化数据”。

在智能逻辑层面, 本系统并未追求显式输出冗长的 Chain-of-Thought, 而是采用“Orchestration as CoT”的工程化思路: 通过流程拆解、阶段性约束与末端校验, 将推理过程隐式编码在系统结构之中。这种设计既保留了大模型在语义联想与跨域关联方面的能力, 又避免了其在结构生成与事实一致性方面的不稳定性直接暴露给上游系统。

从实践经验来看, 本项目的核心收获并不在于“调用了多强的模型”, 而在于验证了一条

重要工程结论：只有当大语言模型被纳入一套明确的工程约束体系之中，它才能真正成为可依赖的软件系统组件，而不是一个不可预测的黑箱生成器。 这一经验对于今后构建更复杂的智能体系统、工具调用型 **Agent** 或多智能体协作系统，具有直接的工程借鉴意义。